

Neural Networks Essentials

Convolutional Neural Networks

Denis C. Ilie-Ablachim

March 18, 2026

Table of Contents

Introduction

Core Concepts of CNNs

- Convolutional Layers

- Pooling (Downsampling)

- Activation Functions

- Fully Connected Layers

Weight Initialization

Architectural Considerations

Training and Practical Tips

Advanced Topics

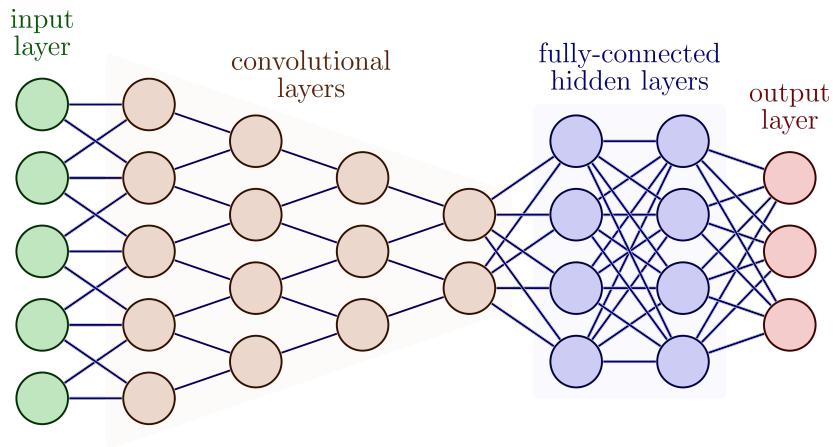
Applications

Conclusion

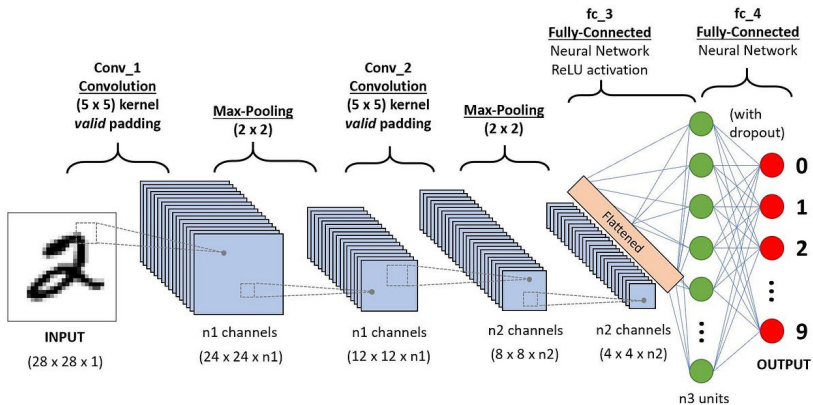
Motivation and Context

- ▶ Convolutional Neural Networks (CNNs) are specialized NN architectures designed for grid-like data (e.g., images, audio spectrograms).
- ▶ Inspired by the visual cortex, CNNs leverage spatial invariance to detect features in a local, translation-invariant manner.
- ▶ Widely applied in image recognition, object detection, natural language processing, and more.
- ▶ We will cover the basics of CNN architecture, the convolution operation, pooling, weight initialization, and practical design tips.

Convolutional Neural Network



How CNN work?



Convolution Operation (Overview)

Key Ideas:

- ▶ **Local Receptive Fields (Filters/Kernels):** Each filter slides across the input, extracting local features.
- ▶ **Sparsity of Connections:** Filters are much smaller than the input, drastically reducing parameters.
- ▶ **Parameter Sharing:** The same filter is reused across multiple spatial locations.
- ▶ **Strides and Padding:** Control output feature map size and boundary effects.

Benefits:

- ▶ Reduced computational complexity and memory.
- ▶ Translation equivariance: learn features independent of location.
- ▶ Natural fit for images, audio, time-series data.

Convolution Operation (Detailed)

Mathematical Definition (2D case):

$$Z(i,j) = (X * K)(i,j) = \sum_m \sum_n X(i+m, j+n) \cdot K(m,n)$$

Example Visualization:

- ▶ Imagine a 3×3 filter scanning an image.
- ▶ At each position, multiply elementwise and sum up to get a single output value.
- ▶ This output value represents how strongly the filter “matches” the local region.

Feature Extraction:

- ▶ **Edge Detection Filter:** A kernel might highlight vertical or horizontal edges by responding strongly where pixel intensity changes sharply.
- ▶ By stacking multiple filters, we learn different features (edges, corners, textures, shapes, etc.).

Max Pooling

2×2 Max Pooling with Stride 2:

1. Partition the feature map into non-overlapping 2×2 regions.
2. For each region, take the maximum value.
3. Outputs a smaller feature map (height/width halved if stride=2).

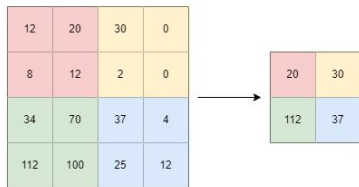


Figure: Illustration of max pooling operation on a 4×4 region.

Benefits:

- ▶ Dimensionality reduction and computational savings.
- ▶ Translation invariance: The exact location of a high value within the 2×2 window is not as important.

Activation Functions

- ▶ Commonly use **ReLU** ($\max(0, x)$) to combat vanishing gradients.
- ▶ Other variations: **Leaky ReLU**, **ELU**, etc.
- ▶ Nonlinearity is essential for learning complex features.

Fully Connected Layers

- ▶ After convolution/pooling, feature maps are flattened and passed to one or more fully connected layers.
- ▶ The final layer often uses a softmax for classification.

He and Xavier Initialization

Key Idea: Proper initialization maintains stable signal magnitudes across layers.

Xavier/Glorot Initialization

- ▶ Typically used when activations are symmetric around zero (e.g. sigmoid, tanh).
- ▶ Variance scales with $\frac{2}{n_{in} + n_{out}}$.

He Initialization

- ▶ Often used for ReLU variants.
- ▶ Variance scales with $\frac{2}{n_{in}}$.

Why?

- ▶ Prevents gradients and activations from exploding or vanishing too quickly.
- ▶ Improves convergence speed in deep networks.

Example Architectures

- ▶ **LeNet** (1990s): Early CNN for digit recognition.
- ▶ **AlexNet** (2012): Popularized deep CNNs via ImageNet.
- ▶ Modern variants: deeper (ResNet, DenseNet), efficient (MobileNet), or attention-based (Vision Transformers, etc.).

Practical Considerations for CNN Development

- ▶ **Kernel Size:** Often 3×3 or 5×5 . Smaller kernels stack well to simulate larger receptive fields.
- ▶ **Activation Function:** ReLU is standard; alternatives (Leaky ReLU, ELU) might help negative gradients.
- ▶ **Regularization Methods:** Dropout, weight decay, batch normalization, data augmentation.
- ▶ **Optimization Strategies:** SGD or Adam, with learning rate scheduling. Carefully tune hyperparameters!
- ▶ **Depth & Width:** Deeper networks can learn richer features, but risk vanishing gradients if poorly designed.

Training a CNN

- ▶ **Forward/Backward Propagation:** Convolutions + pooling + activations + fully connected layers.
- ▶ **Regularization:** Dropout, batch normalization, **data augmentation**.
- ▶ **Initialization:** (He or Xavier) to stabilize gradients.
- ▶ **Optimization:** SGD or Adam + learning rate scheduling.

Advanced Topics

- ▶ **Residual Connections (ResNet):** Skip connections to mitigate gradient vanishing.
- ▶ **Dilated Convolutions:** Expand receptive field without increasing parameters.
- ▶ **Depthwise/Separable Convolutions:** Reduce computation for mobile and embedded devices.
- ▶ **Transposed Convolutions:** Used for upsampling in generative models (GANs) or segmentation.

Selected Applications

- ▶ **Image Classification:** Recognizing objects (e.g. ImageNet).
- ▶ **Object Detection & Localization:** e.g. Faster R-CNN, YOLO, SSD.
- ▶ **Semantic Segmentation:** Pixel-level labeling (Fully Convolutional Networks).
- ▶ **Medical Imaging:** Tumor detection, volumetric segmentation.
- ▶ **Audio Processing:** Speech recognition, music analysis (spectrogram-based).

Key Takeaways

- ▶ Convolution + pooling effectively reduce parameters and capture spatial/temporal structure.
- ▶ Proper initialization (He/Xavier) prevents exploding/vanishing gradients.
- ▶ Deeper networks + advanced regularization = state-of-the-art results.
- ▶ CNNs have powered major breakthroughs in computer vision and beyond.